

QPU Bench

Open Source Quantum Computing Benchmark Framework

Mark Nicholas Jones · mark@mq.s.dk

2026

Molecular Quantum Solutions (MQS)

Framework Overview

What is QPUBench?

Modality-agnostic quantum benchmark framework

- Separates *what* you benchmark (schemas) from *how* execution happens (adapters) and *how results are stored* (stores)
- Typed **Pydantic v2** schema layer — zero quantum SDK dependencies in core
- Gate-based circuits, MBQC on FPGA, algorithm classes for quantum chemistry (VQA, KQD, SKQD, QSE, QPE), evolutionary + constrained circuit synthesis, and error-mitigated runs all share one BenchmarkRecord
- Schema **v2.7.0** — 36 modules, language-agnostic JSON output

Core invariant

qpubench itself never imports from any quantum library. Only adapters do.

Architecture

Core layer (*no SDK dependencies*)

- schemas/ — 36 Pydantic v2 modules
- runner.py — BenchmarkRunner
- store.py — NDJSONStore, ParquetStore

Integration layer (*your project*)

- backends/ — adapter protocols + stubs
- integrations/ — copy into your project
- examples/ — runnable demos



Protocol vs. stub — BackendAdapter is the interface contract (spec/validate/run), no SDK code. A **stub** fakes that contract; real adapters (Aer, IBM, IQM, Braket) implement it with real SDK calls, verified end-to-end in this repo's tests. Swapping a stub for a real adapter needs zero code changes elsewhere.

Key Design Principles

Principle	Implementation
Schema-first	Pydantic v2 is the stable contract — all records are valid JSON
No SDK lock-in	Core never imports any quantum library
Three adapter protocols	BackendAdapter · AlgorithmAdapter · ErrorMitigationAdapter
Append-only stores	NDJSON and Parquet for reproducible sweeps
Language-agnostic	Any language can parse the NDJSON output

Installation

```
# Minimal – schemas + stubs, no quantum SDK required
```

```
pip install .
```

```
# With optional backends
```

```
pip install ".[qiskit]"      # Qiskit Aer + IBM Quantum Runtime V2
```

```
pip install ".[storage]"    # Parquet store (pyarrow + pandas)
```

```
pip install ".[s3]"         # S3 store (boto3) – AWS S3 / MinIO / HF buckets
```

```
pip install ".[all]"        # everything on PyPI
```

Package managers: pip · uv · Poetry 2 · conda

Credentials: via .env file and BackendSpec.auth (never hardcoded)

Schema Layer

Core Schema Modules (1/3)

All rows are computing-model/qubit-modality **all** / **all** — these are the framework's own foundational types, not vendor-specific.

Module	Key types
primitives	ComputingModel, QubitModality, CircuitFormat, PauliLabel, ComplexNumber
circuit	CircuitSpec — from_openqasm3(), bind(), is_parametric()
observable	SparsePauliObservable, PauliTerm
backend	BackendSpec — 28 factory constructors

Core Schema Modules (2/3)

Module	Key types
execution	ExecutionOptions, AlgorithmSpec, ZNEConfig
advantage	QuantumAdvantageRecord — multi-org registry, unprefixed
result	QuantumResult, ExpectationResult, ShotResult
record	BenchmarkRecord, VQAConfig

Core Schema Modules (3/3)

Module	Key types
reaction	ReactionCoordinateSpec, ReactionPathResult — ties a sweep into one reaction path / PES
optimizer_catalog	MINIMIZER_CATALOG, STOPPING_CRITERION_CATALOG — lookup over AdaptVQEConfig fields
hamiltonian_library	HamiltonianSource, HamiltonianLibraryRecord — metadata for real Hamiltonians (PennyLane qchem, HamLib, ab initio)
contraction_path	ContractionPathStrategy/Config/Result — real quimb + cotengra tensor-network contraction

Domain-Specific Schema Modules (1/2)

Named `<org>_<package>.py` — who maintains it + what it's called:

Module	Computing model (qubit modality)
<code>johnrscott_mbqc_fpga</code>	MBQC-FPGA — bit-exact 16-bit program word
<code>dtu_photonic</code>	LOQC / FBQC (photonic) — Fock states, HOM, photonic VQE/analog
<code>dtu_gbs</code>	GBS (photonic) — hafnian, vibronic, TDM/Borealis
<code>microsoft_qdk</code>	Gate-based (QPE technique) — SCF → active space → resource estimation
<code>mqsdk_qse</code>	Gate-based (KQD technique) — Krylov Quantum Diagonalization / SQD

Domain-Specific Schema Modules (2/2)

Module	Computing model (qubit modality)
<code>molssi_qcschema</code>	all / all — QCSchema / QCElemental / PennyLane interoperability
<code>quera_bloqade</code>	Adiabatic (neutral atom) — Bloqade / Aquila AHS
<code>erikkjellgren_slowquant</code>	Gate-based — SlowQuant UCC/VQE ansatz, SCF, linear response
<code>classiq</code>	Gate-based — Classiq synthesis + chemistry/QAOA, harmonized with <code>mqsdk_xenakis</code>
<code>pyscf</code>	Gate-based (chemistry) — molecules/cells, mean-field/DFT, PCM solvation, ERI builder, orbital optimizer (all real); DMET/projection embedding (schema-only)
<code>polarizable_embedding</code>	Gate-based (chemistry) — CPPE + PyFraME real potfile format, via <code>pyscf.solvent.PE</code>

Computing Model × Qubit Modality

Two **independent, orthogonal** axes — not one flat enum. Many-to-many: some paradigms run on several QPU modalities, but not every pairing exists.

ComputingModel (paradigm)

- GATE_BASED — circuit model
- MBQC — measurement-based
- FUSION_BASED — FBQC resource states + fusion
- ADIABATIC — continuous-time evolution (AHS)
- ANNEALING — quantum annealing
- GBS — Gaussian Boson Sampling
- SAMPLING — general sampling paradigms

QubitModality (QPU modality)

- SUPERCONDUCTING — IBM, IQM
- TRAPPED_ION — Quantinuum, IonQ
- NEUTRAL_ATOM — QuEra
- PHOTONIC — Quandela, Xanadu
- SILICON_SPIN — Quantum Motion

GATE_BASED alone spans superconducting, trapped-ion, silicon-spin, and photonic hardware — GBS/FUSION_BASED are photonic-only today. QPE/KQD are techniques on top of GATE_BASED, not separate paradigms. Every combination shares the same BenchmarkRecord.

CircuitSpec — One Type for All Circuits

```
# Gate-based – read a .qasm file straight into `serialized`
source = pathlib.Path("bell_state.qasm").read_text()
circuit = CircuitSpec(num_qubits=2, serialized=source,      # OpenQASM 2.0
                      observables=[...])

# OpenQASM 3.0 – same file-read pattern
circuit = CircuitSpec.from_openqasm3(
    pathlib.Path("bell_state.qasm3").read_text(), num_qubits=2)

# Parametric VQE – bind() returns a new copy
ansatz = CircuitSpec(num_qubits=2, parameters=["theta"], ...)
bound   = ansatz.bind({"theta": 1.2566})
```

Real Hamiltonian Sources

Three ways to get a real molecule's qubit Hamiltonian as a `SparsePauliObservable` (all in `hamiltonian_sources.*`)
— drops into any Estimator path unmodified:

Source	Function	What it gives you
HamLib Chemistry	<code>load_hamlib_chemistry()</code>	HDF5 (NERSC), 4 encodings, no ref. energies
PennyLane qchem	<code>load_pennylane_qchem()</code>	Dataset service, ships <code>fci_energy/vqe_energy</code>
Ab initio (new)	<code>build_qubit_hamiltonian()</code>	<i>Any</i> geometry — PySCF HF + active-space + JW

```
obs, record = build_qubit_hamiltonian(  
    geometry, basis="sto-3g", active_electrons=2, active_orbitals=2)
```

Verified: HamLib's H₂ Hamiltonian through the *unmodified* ADAPT-VQE engine converges to -1.13146 Ha, matching dense diagonalization to 7e-15 — `jordan_wigner()`'s `str()` is byte-identical to HamLib's stored strings, one parser serves both.

Closing the Last 4 qrunch Guide Gaps

The final 4 “no qpubench mechanism” guides — all real, closed with quimb+cotengra (per direct steer) or PySCF (already a dependency):

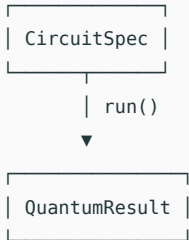
Guide	Mechanism	Verified
Contraction Path Finder	quimb.tensor.Circuit + cotengra.HyperOptimizer	4/4 strategies real; NONE costs more (180224 vs 248)
ERI Builder	mol.intor('int2e') vs mf.density_fit() (RI/DF)	RI matches standard ERIs to 2e-5 Ha (H2O/cc-pVDZ)
FAST / Brute Force Gate Selector	GateSelector protocol — gradient screen vs full re-opt	Both converge to exact H2 ground state
Orbital Optimizer	Newton (CASSCF) / Simple (kappa-rotation + CASCI) / Basin-Hopping	All 3 agree on H2/6-31G — Newton vs Basin-Hopping: 3e-9 Ha

A real bug found along the way: quimb’s `contraction_info(optimize=False)` raises `TypeError` (cotengra’s dispatch mistakes `False` for a custom path-finder callable) — worked around via `opt_einsum.contract_path()` directly.

Adapters

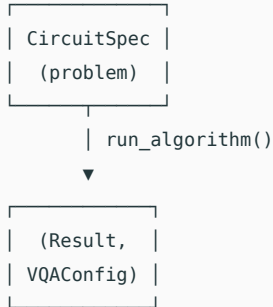
Three Adapter Protocols

BackendAdapter *Circuit in, result out*



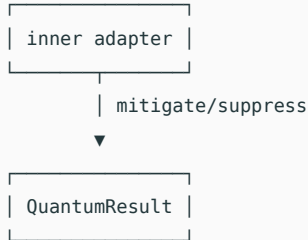
Aer · Qrack · IBM IQM ·
MBQC-FPGA

AlgorithmAdapter *Problem spec in*



QForte · OpenFermion

ErrorMitigationAdapter *Wraps a BackendAdapter*



Fire Opal · Mitiq Haiqu ·
ParityQC

BenchmarkRunner dispatches automatically — ErrorMitigationAdapter registers like any BackendAdapter.

BackendAdapter — Estimator vs Sampler

```
class MyBackendAdapter:
    @property
    def spec(self) -> BackendSpec: ...    # hardware description
    def validate(self, circuit) -> list[str]: ...
```

Estimator path — circuit.observables set

```
def run(self, circuit, options):
    evs = [measure_expectation(circuit, obs)
           for obs in circuit.observables]
    return QuantumResult(
        expectation_values=evs, ...)
```

VQE / QAOA objective values

Sampler path — no observables

```
def run(self, circuit, options):
    counts = sample_bitstrings(
        circuit, options.shots)
    return QuantumResult(
        shots=ShotResult(
            ..., counts=counts), ...)
```

Raw bitstring / shot counts

Rule: SDK imports go inside the method that uses them — never at module level.

AlgorithmAdapter (QForte example)

```
class QForteAlgorithmAdapter:
```

```
    validate_problem()
```

```
def validate_problem(self, circuit):
    if circuit.format != \
        CircuitFormat.MOLECULE_JSON:
        return [
            f"Expected MOLECULE_JSON, "
            f"got {circuit.format!r}"
        ]
    return []
```

```
    run_algorithm()
```

```
def run_algorithm(self, circuit, options):
    import qforte as qf    # deferred import
    cfg = options.adapt_vqe_config
    mol = qf.system_factory(...)
    alg = qf.ADAPTVQE(mol, ...)
    alg.run(pool_type=cfg.pool_type,
            optimizer=cfg.optimizer)
    result = QuantumResult(
        qforte_result=QForteRunResult(...),
        expectation_values=[...])
    return result, VQAConfig(...)
```

AlgorithmFamily — Switch Implementations Freely

AlgorithmSpec carries **identity only** (name + family) — hyperparameters live in one shared config every adapter for that family accepts:

```
config = AdaptVQEConfig(pool_type="SD", optimizer="BFGS")
opts = ExecutionOptions(algorithm_spec=AlgorithmSpec(
    name="ADAPT_VQE", family=AlgorithmFamily.ADAPT_VQE),
    adapt_vqe_config=config)

runner.run(mol, "qforte", opts)           # native C++
runner.run(mol, "ibm_qiskit_adapt_vqe", opts)  # from-scratch Qiskit
runner.run(mol, "microsoft_qdk_adapt_vqe", opts) # QDK / Azure Quantum
```

Three adapters implement AlgorithmFamily.ADAPT_VQE: integrations/qforte/ (QForte's native C++ statevector), and ibm_qiskit_adapt_vqe/ / microsoft_qdk_adapt_vqe/ (thin wrappers over the pure-Python generic_adapt_vqe/ engine — no vendor SDK needed).

Xenakis ↔ Classiq — Two Optimization Strategies

Same underlying problem, two strategies — both converge on `CircuitSpec`.

	Xenakis	Classiq
Strategy	GA search	Constrained synthesis
Objective	Soft penalty ($\lambda_{\text{depth}}, \lambda_{2q}$)	Hard bound + objective
Cost	Generations $\times$ population	One solve (duration, s)
Molecule	XenakisMolecule (coords-first)	ClassiqMoleculeSpec (atoms-first)

```
classiq_mol = ClassiqMoleculeSpec.from_xenakis_molecule(xenakis_mol)
cmp = CircuitOptimizationComparison(ga_result=ga_run, classiq_result=synth)
cmp.depth_delta          # GA best-genome depth – Classiq depth
cmp.search_cost_label    # "GA: 40 generations vs Classiq: 2.30s synthesis"
```

Error Mitigation Providers

One module per vendor, one shared `ErrorMitigationStrategy` enum in `primitives.py`:

Provider	Schema module	Technique
Q-CTRL Fire Opal Mitiq	<code>qctrl_fire_opal</code> <code>unitaryfund_mitiq</code>	Closed-box noise suppression ZNE · PEC · CDR · REM · DDD
Haiqu Rivet ParityQC	<code>haiqu_rivet</code> <code>parityqc_parityqc</code>	HW-aware transpilation caching Parity encoding (QUBO/HCBO/Ising)
QMatter Quantum Motion	<code>qmatter_qmatter</code> <code>quantum_motion_hardware</code>	Quantum problem compression CMOS spin-qubit characterisation (<i>hardware vendor, not error mitigation</i>)
QESem (Qedma)	<code>qedma_qesem</code>	QET noise scaling + characterisation

Running on Real Hardware — IBM, IQM & AWS Braket

All three adapters are **real implementations, verified end-to-end** — Aer/Braket run with no credentials needed (BraketLocalBackend); IBM/IQM transpile+run logic verified against bundled fake backends (tests/test_backend_adapters.py). `opts = ExecutionOptions(shots=4096)` — shared by all three below.

IBM Quantum Runtime V2

(qiskit-ibm-runtime)

```
adapter = IBMAdapter(  
    "ibm_torino",  
    execution_mode=  
        IBMExecutionMode.SESSION,  
)  
runner.register(adapter, "ibm")  
runner.run(circuit, "ibm", opts)
```

Estimator/Sampler auto-chosen;
IBMRuntimeRecord carries job/session
IDs.

IQM Resonance cloud (iqm-client + qiskit-iqm)

```
adapter = IQMAdapter(  
    "garnet", # 20Q  
    token=os.environ[  
        "IQM_TOKEN"],  
)  
runner.register(adapter, "iqm")  
runner.run(circuit, "iqm", opts)
```

Star topology: PRX/CZ + MOVE via
COMPR1 — no SWAP routing.

AWS Braket (Rigetti, IonQ, OQC, SV1/DM1/TN1)

```
spec = BackendSpec.braket(  
    "arn:aws:braket:...:"  
    "device/qpu/rigetti/"  
    "Ankaa-3",  
    s3_bucket_ref="MY_BUCKET",  
)  
runner.register(  
    BraketAdapter(  
        spec.auth["device_arn"],  
        s3_bucket_ref="MY_BUCKET",  
    ), "braket")  
runner.run(circuit, "braket", opts)
```

Every task needs an S3 result
location; `AwsDevice.run()` is async. 21

IBM Quantum Advantage Tracker

BenchmarkRecord.advantage holds a QuantumAdvantageRecord:

```
record = runner.run(circuit, "ibm_torino", options)
record = record.model_copy(update={"advantage": QuantumAdvantageRecord(
    experiment_type=AdvantageExperimentType.OBSERVABLE_ESTIMATION,
    circuit_name="Loschmidt-echo-70q",
    num_qubits=70,
    backend_name="ibm_boston",
    floquet_layers=20,
    observable_value=0.327,
    observable_error_bound=0.012,
    classical_method=ClassicalComparisonMethod.TENSOR_NETWORK,
    coupling_params={"b": 1.0, "delta": 0.5},
    submission_url="https://github.com/quantum-advantage-tracker/...",
}))
```

Execution & Storage

BenchmarkRunner

```
runner = BenchmarkRunner(store=NDJSONStore(Path("results.ndjson")))
runner.register(AerAdapter(), name="aer")
runner.register(QForteAlgorithmAdapter(), name="qforte")
record = runner.run(circuit, "aer", ExecutionOptions(shots=4096)) # single run

# Cartesian product sweep: circuits x backends x options
records = runner.sweep(
    circuits=[circuit_h2, circuit_lih], backend_names=["aer", "qforte"],
    options_list=[ExecutionOptions(shots=1024), ExecutionOptions(shots=8192)],
    run_id="vqe-sweep-001",
)
runner.add_hook(lambda rec: log_energy(rec.vqa.final_eigenvalue)) # post-run hook
```

Structured Logging — BenchmarkLogger

Built directly on `BenchmarkRunner.add_hook()` — real levels, handlers, and formatters, not one flat log line:

```
from qpubench import BenchmarkLogger
bench_logger = BenchmarkLogger(name="qpubench.benchmark")
bench_logger.attach(runner)          # wires into add_hook() internally
record = runner.run(circuit, "aer", ExecutionOptions(shots=4096))
# {"experiment_id": "...", "backend": "aer_statevector", "status": "succeeded", ...}
```

Levels	Handlers	Formatters
SUCCEEDED → INFO, FAILED → ERROR	any logging.Handler	JSONFormatter (default), swappable

Data Persistence

NDJSONStore — zero dependencies, append-only, grep-able

```
store = NDJSONStore(Path("results/run1.ndjson"))
store.save(record)

record = store.load(experiment_id)           # by UUID
records = store.query(                       # dot-path filters
    backend__name="aer_statevector",
    result__status="succeeded",
)
```

ParquetStore — columnar analytics (pip install "[storage]")

```
store = ParquetStore(Path("results/run1.parquet"))
df = store.to_dataframe()                   # → pandas DataFrame
```

S3Store — AWS S3 & Hugging Face Buckets

One JSON object per record (`{prefix}/{experiment_id}.json`) — no shared file, so concurrent writers never race. `pip install "qpubench[s3]"`.

AWS S3 (or any S3-compatible endpoint)

```
from qpubench import S3Store

store = S3Store(
    "my-results-bucket",
    prefix="sweeps/2026-07-06",
    region_name="eu-west-1",
)
```

Any `boto3.client("s3", ...)` kwarg works — same code path covers MinIO too.

Same `ResultStore` protocol as `NDJSONStore`: `save()` / `load()` / `query()` / `all()`.

Hugging Face Storage Buckets

```
store = S3Store.huggingface(
    bucket="my-training-bucket",
    namespace="my-username",
    access_key_id="HFAK...",
    secret_access_key="...",
)
```

`.huggingface()` sets the gateway's required Config — path-style, single region, V2-only listing.

Integrations

Integration Ecosystem — BackendAdapter

Schema modules are named `<org>_<package>.py` — one glance tells you who maintains it and what it's called.

Integration	Schema module
Cebule SDK	<code>mqsdk_cebule</code>
Photochipsim / FBQC	<code>dtu_photonic</code>
DTU-GBS / photonic_QC	<code>dtu_gbs</code>
QESEM (Qedma)	<code>qedma_qesem</code>
Bloqade / Aquila	<code>quera_bloqade</code>
IQM Resonance (garnet · deneb · sirius)	—
Quantum Motion CMOS spin-qubit	<code>quantum_motion_hardware</code>

Integration Ecosystem — AlgorithmAdapter

Integration	Schema module
Xenakis GA circuit search	mqsdk_xenakis
QForté (ADAPT-VQE, UCCN-VQE, ...)	evangelistalab_qforte
Classiq circuit synthesis	classiq
ExcitationSolve (Fourier VQE)	dlr_excitation_solve
GSOpt (ground-state benchmark)	bestquark_gsopt
QDK / QuNorth chemistry	microsoft_qdk
QSE / KQD	mqsdk_qse
SlowQuant UCC/VQE (integrations/slowquant/, real code, not on PyPI)	erikkjellgren_slowquant

Integration Ecosystem — ErrorMitigationAdapter

One module per vendor now — previously all bundled into a single `error_mitigation` grab-bag (including a hardware vendor miscategorized as “error mitigation”).

Integration	Schema module
Q-CTRL Fire Opal	<code>qctrl_fire_opal</code>
Mitiq (ZNE · PEC · CDR · REM · DDD)	<code>unitaryfund_mitiq</code>
Haiqu Rivet transpiler	<code>haiqu_rivet</code>
ParityQC parity encoding	<code>parityqc_parityqc</code>
QMatter problem compression	<code>qmatter_qmatter</code>

Orchestration — Kubeflow Pipelines

Different from every integration above: not a schema bridge, but how algorithmic steps get **run** as scheduled, dashboard-visible components.

Two Kubeflow SDKs, two roles:

SDK	Role here
kfp (Pipelines)	Owns the DAG + Central Dashboard — every step is a native <code>@dsl.component</code>
Unified SDK	Narrow, opt-in (<code>TrainerClient/OptimizerClient</code>) — called <i>inside</i> one component body only, for multi-node dispatch or Katib search

Why not the unified SDK as the top-level abstraction?

Its own docs list Pipelines support as "Planned," not shipped — and `TrainerClient/CustomTrainer` are shaped around ML training, a frame that fits nothing here. `kfp` already gives DAG construction, branching, artifacts, and caching natively.

Component Taxonomy

Every pipeline decomposes into two kinds of kfp node — same shape, different resource profile:

Kind	Shape	Examples
Transform	pydantic-in → pydantic-out, no BackendAdapter	SCF, active-space selection, Cebule MOL_MAP/TN_QC_OPT
Execution	BenchmarkRunner.run() against a registered adapter	GPU sim (Qrack) · real QPU (IBM, IQM, QESEM)

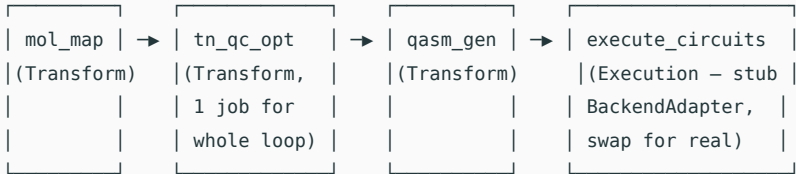
Kubeflow can't reach a QPU directly — an Execution component targeting real hardware does what BackendAdapter.run() already does: call the vendor SDK, submit to *their* queue, poll, return.

Kubeflow's role is a scheduled, credentialed place to host that client, not compute.

Iterative loops stay one job — TN_QC_OPT's n_iterations, ADAPT-VQE's macro/micro-iterations never explode into one DAG node per iteration.

Worked Example — Cebule Task Chain as a kfp DAG

integrations/kubeflow/ — four DAG nodes, matching the Cebule SDK's own documented task order:



Verified by actually compiling against kfp==2.16.1, not just reading the API — surfaced a real gotcha: from `__future__` import `annotations` breaks kfp's component introspection (it needs live type objects, not PEP 563 strings), unlike the rest of this repo's convention.

Credentials injected via kfp-kubernetes's `use_secret_as_env` — never as plain pipeline parameters (those land in the dashboard's recorded run history). See `docs/integrations/kubeflow.md`.

Cross-Framework Compatibility

Pauli encoding — explicit converters prevent silent convention bugs:

```
# Qrack Q# convention: I=0, X=1, Z=2, Y=3 (non-sequential!)
PauliLabel.Z.to_qrack_int()          # → 2 (not 3)
PauliLabel.Y.to_qrack_int()          # → 3 (not 2)

# Qiskit C API QkBitTerm bit-packed encoding
PauliLabel.X.to_qiskit_c_bit_term()  # → 0b0010

# MBQC byproduct register: bit 0 = Z, bit 1 = X (reversed vs gate-based!)
```

Complex numbers: stored as {"re": 1.0, "im": 0.5} — valid JSON without Python eval

QCSchema / QCElemental / PennyLane interoperability via the `molssi_qcschema` module

Code Quality

Python Quality Toolchain

```
ruff check src/ tests/    # lint (line-length 100, target py311)
ruff format src/ tests/   # format
mypy src/                 # static types (strict = true)
pytest tests/ -q          # 259 tests – SDK-dependent ones skip cleanly
                          # if qiskit/braket/iqm/openfermion aren't installed
```

Tool	Role
Pydantic v2	Runtime-validated schemas — field types enforced at parse time
ruff	Linting + auto-formatting, replaces flake8/isort/black
mypy strict	Full type coverage — Any only where unavoidable
pytest	Schema-only test suite runs with pip install . alone
ponytail	Agent code-quality enforcement via decision ladder

Case Study — qrunch Guide Parity

Can qpubench Reproduce a Competitor's Guide Catalog? (1/2)

Audited every [qrunch](#) how-to guide, demo, and tutorial (Kvantify's VQE chemistry package) against qpubench — **without installing qrunch**, with **FAST-VQE/BEAST-VQE** substituted by **ADAPT-VQE** everywhere (qrunch's algorithms are proprietary).

	Guides (22)	Demos (6)	Tutorials (5)
Yes	22	3	0
Partial	0	3	5
No	0	0	0

Four passes, each rechecked against real sources (docs.mqs.dk, Cebule SDK source, PySCF): pass 1-2 closed embedding gaps (**InQuanto checked, not required**) and 2 more via `pip install pyscf`. Pass 3 made **backend adapters real implementations** (not `NotImplementedError` stubs), Active Space run real PySCF AVAS, and added real Minimizer/Logger/Polarizable-Embedding mechanisms. **Pass 4 closed the last 4 Guides** — Contraction Path Finder, ERI Builder, Gate Selectors, Orbital Optimizer — **Guides are now 22/22 “Yes”**.

Can qpubench Reproduce a Competitor's Guide Catalog? (2/2)

Tutorials stay 5/5 “Partial” numerically, but 3 upgraded from an *invented toy Hamiltonian* to a *real ab initio* one on the actual qcrunch molecule/mechanism — checked the real qcrunch tutorial notebooks directly (github.com/Kvantify/qcrunch_tutorials), not guessed: butyronitrile's real C#N bond, a real carboxylate-mediated SN2 (matching the dehalogenase notebook's own embedding), and cathepsin K's real nitrile-cysteine covalent mechanism (not the Michael-addition guess first assumed).

25+ new runnable examples, plus `reaction.py`, `AWS Braket`, `schemas/pyscf.py`, `schemas/polarizable_embedding.py`, `schemas/optimizer_catalog.py`, `schemas/contraction_path.py`, `observability.py`, a real `integrations/slowquant/ adapter`, `hamiltonian_sources/` (HamLib + PennyLane + ab initio), and `tensor_network/contraction_path.py` + a refactored, swappable GateSelector.

Methodology — Honest by Construction

Two temptations avoided:

No fabricated chemistry

Illustrative qubit Hamiltonians (`toy_hamiltonians.py`) stay explicitly labeled, never presented as physically accurate. Real molecules: `qforte_vqe_benchmark.py` (He/cc-pVDZ), and now `hamiltonian_sources/` — real ab initio Hamiltonians replaced the toy ones in 3 tutorials, checked against real crunch notebooks rather than guessed stand-ins.

No random numbers standing in for physics

`StubGateAdapter` returns *random* expectation values — useless as an ADAPT-VQE energy oracle (the optimizer would chase noise). Built `ToyStatevectorAdapter`: a real, small, dense-matrix statevector simulator instead.

No fabricated API calls either

`PsiEmbed`, `libDMET`, and `SlowQuant` aren't on PyPI. Their adapters call each package's *actual documented API* — checked field-by-field against real GitHub source, not guessed — behind an `ImportError` guard: real code, honestly unexecuted until installed from source.

Summary

Getting Started

```
from qpubench import (
    BenchmarkRunner, NDJSONStore, StubGateAdapter,
    CircuitSpec, ExecutionOptions, SparsePauliObservable,
)
import pathlib

circuit = CircuitSpec(
    num_qubits=2, serialized=pathlib.Path("bell_state.qasm").read_text(),
    observables=[SparsePauliObservable(num_qubits=2, terms=[...])],
)
runner = BenchmarkRunner(store=NDJSONStore(pathlib.Path("results.ndjson")))
runner.register(StubGateAdapter(seed=42), name="stub")
record = runner.run(circuit, "stub", ExecutionOptions(shots=4096))
print(f"<ZZ> = {record.result.expectation_values[0].value:.4f}")
```

Roadmap & Links

Repository: github.com/mqsdk/qpubench

Schema v2.7.0 — 36 modules · 7 computing models × 5 qubit modalities · zero quantum SDK deps in core

Contributing — writing an adapter takes ~30 lines:

```
cp integrations/template/backend_adapter_template.py my_adapter.py
# fill the three TODOs: spec, validate(), run()
```

See [INTEGRATION_GUIDE.md](#) and [AGENTS.md](#) for the full development workflow.

Contact: contact@mqs.dk